(V) (e) el refactoring form template method permite eliminar el code smell de código duplicado form template method: refactoring q' mueve pasos a una superclase.

(F) (f) las clases de un framework son dependientes unas de las otras, no puedo x lo gral reutiliza una sola clase de un framework independiente

(V) (g) un framework de caja negra se reúsa x subclasificax

(F) (h) si un método tiene 3 o 4 parámetros es un mal olor y hay varios refactoring p/ aplicar.

(V) long parameter list: larga lista de parámetros, puedo aplicar replace parameter with method, preserve whole object, introduce parameter object.

(F) (i) el patrón null object es uno de los conocidos como "wrapper" es un bad smell.

(V) (j) el patrón template method busca minimizar los métodos q' implementan las subclases

(V) (k) el patrón proxy puede ser usado puede ser usado p/ retrasar las búsquedas

(F) (l) el patrón state solo puede ser implementado en objetos q' tienen datos pero no comportamiento.

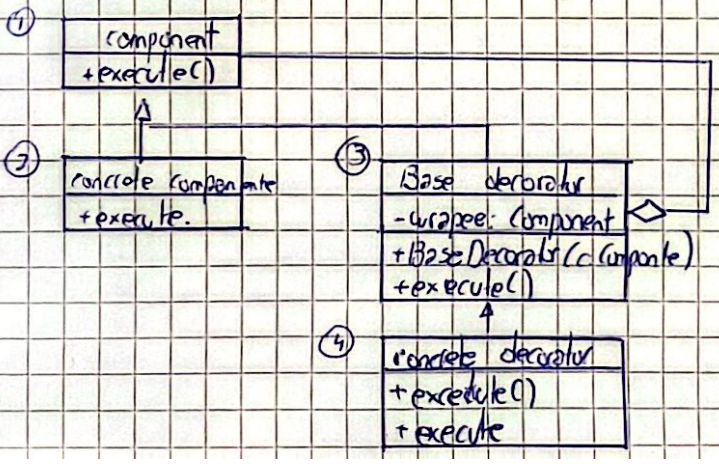
(V) (m) el patrón adapter desacopla la interface de la implementax de esa interface.

(7) Los patrones de diseño se documentan siguiendo una estructura/organizax predefinida. ¿Cuál es?.
Un patrón usualmente tiene 4 elementos esenciales.
(1) nombre del patrón: describe d/ una o 2 palabras un problema de diseño junto c/ sus soluciones y consecuencias.
(2) el problema: describe cuándo aplicar el patrón, explicando el problema y el contexto.
(3) la solux: describe los elementos que constituyen el diseño, relaciones, responsabilidades y colaboraciones. La solux no describe un diseño o implementax, sino el patrón como plantilla a aplicar en <> situaciones.
(4) consecuencias: resultados, ventajas e inconvenientes de aplicar este patrón.

(5) Describa el patrón decorator. A modo de ejemplo describa su objetivo, el problema q' resuelve, cuándo usarlo, modelo de clases, consecuencia de su uso, etc.
Patrón de diseño estructural q' permite agregar funcionalidades a objetos colocando esos objetos dentro de objetos encapsuladores q' contienen esas funcionalidades.

Se utilizará p/ agregar responsabilidades a objetos individuales sin afectar a otros o quitar responsabilidades dinámicamente.

consecuencia: mayor flexibilidad q' herencia.
implementax: misma interface el componente y decorator.



① Componente: declara la interfaz común tanto p/ wrappers como p/ objetos envueltos.
② Componente concreto: es una clase de objeto envueltos. Define el comportamiento básico, que los decoradores puedan alterar.
③ base decorador: tiene un campo p/ referenciar un objeto envuelto. El tipo del campo debe declararse como la interfaz del componente p/ q' pueda contener tanto los componentes concretos como los decoradores. La base clase decoradora base delega todas las operaciones al objeto envuelto.
④ concrete decoradors: definen funcionalidades adicionales q' se pueden añadir dinámico' a los componentes. Los decoradores concretos sobreescriben métodos de la clase decoradora base y ejecutan su comportamiento, ya sea antes o después de invocar el método padre.

⑨ ¿Cuándo se aplica template method?
El template method se aplica cuando hay varias clases q' comparten lógica en general, pero c/ una necesita personalizar algunos pasos en específico.
Se usa p/ evitar duplicar de código en subclases o se aplican métodos comunes dentro de un mismo método en la superclase.

⑩ ¿Cuándo se aplica el patrón composite?
Se utiliza cuando se requiere presenta jerarquías parte-todo de objetos. o q' los clientes ignoren diferencia e/ composiciones y objetos individuales.

⑪ ¿Cuándo se aplica el patrón state?
Se aplica cuando el comportamiento de un objeto depende de su estado y puede cambiar durante su ciclo de vida. o es necesario cambiar el comportamiento dinámica'

⑫ ¿Cuándo se aplica el patrón adapter?
~~Se usa cuando tenemos una interfaz incompatible la cual es necesario utilizar~~
Se usa cuando es necesario q' dos interfaces incompatibles las sean. p/ q' puedan trabajar en conjunto.

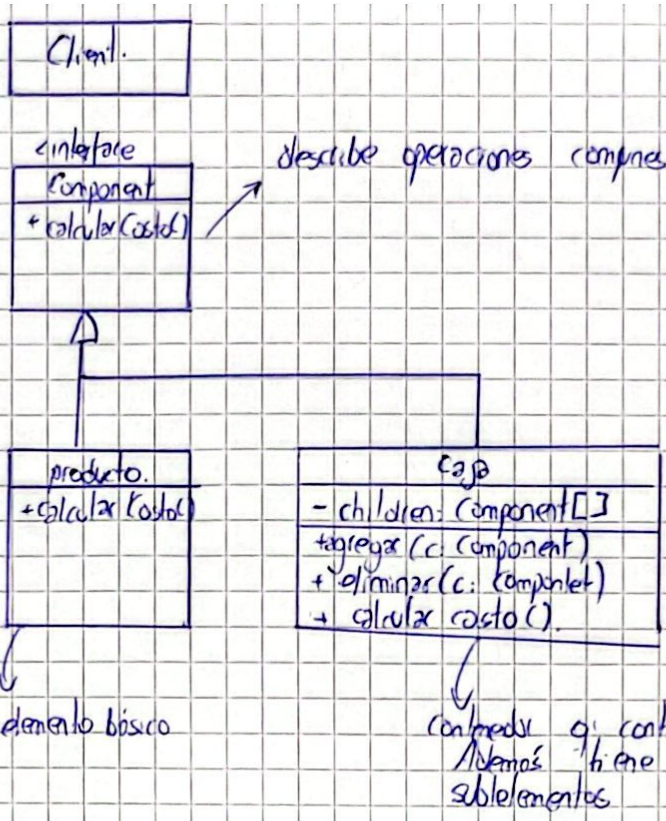
⑬ Identifique semejanzas y diferencias e/ adapter y decorator.
Ambos resultan ser tipo estructurales y wrappers, los cuales decoran el objeto p/ cambiarlo. Las diferencias se presentan en

Decorator	Adapter
Decorator preserva la interfaz del objeto p/ el cliente	Adapter convierte la interfaz del objeto p/ el cliente
Pueden y suelen anidarse	No se anidan.

⑭ Compare patrón decorator c/ strategy
Ambos permiten q' un objeto cambie su funcionalidad dinámico'. Se diferencia es q' el strategy cambia el algoritmo x dentro y el decorator x fuera.

⑮ Describa y ejemplifique el patrón composite
Se usa p/ representar jerarquías. Permite componer objetos en estructuras de árbol y trabajar c/ esta estructura como si fueran objetos individuales.

~~Ejemplo: tengo un pedido q' contiene cajas q' a su vez contiene productos. La solux de trabajar~~
Ejemplo: se tienen 2 tipos de objetos: productos y cajas. Una caja puede contener varios productos así como cierto número de cajas más pequeñas. Estas cajas pequeñas tmb. pueden contener algunos productos o cajas.



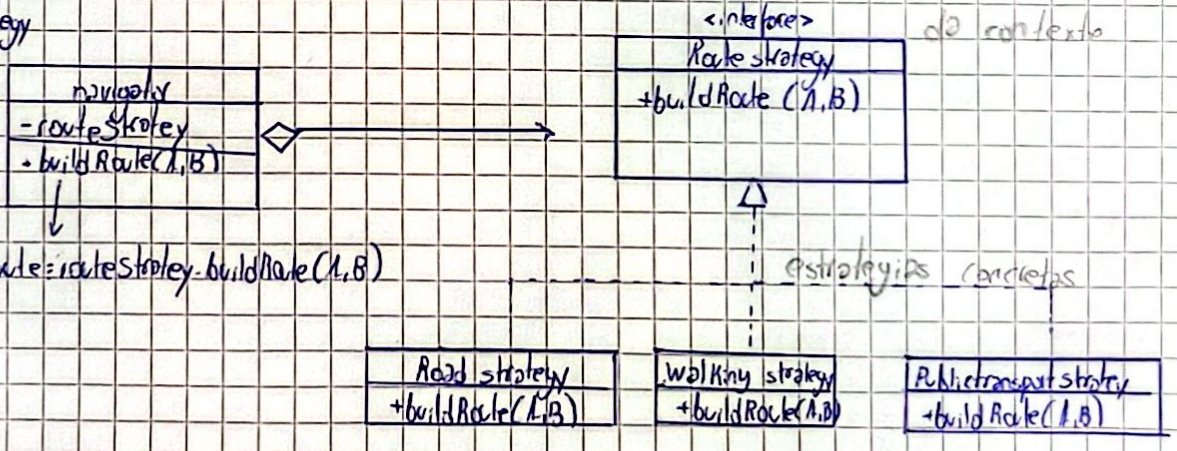
③

⑯ Identifique las semejanzas y diferencias e/ los patrones state y strategy en todos los aspectos que pueda, x ejemplo propósito, modelo de clase, modelo de instancias como tmb. cuestiones de implementax

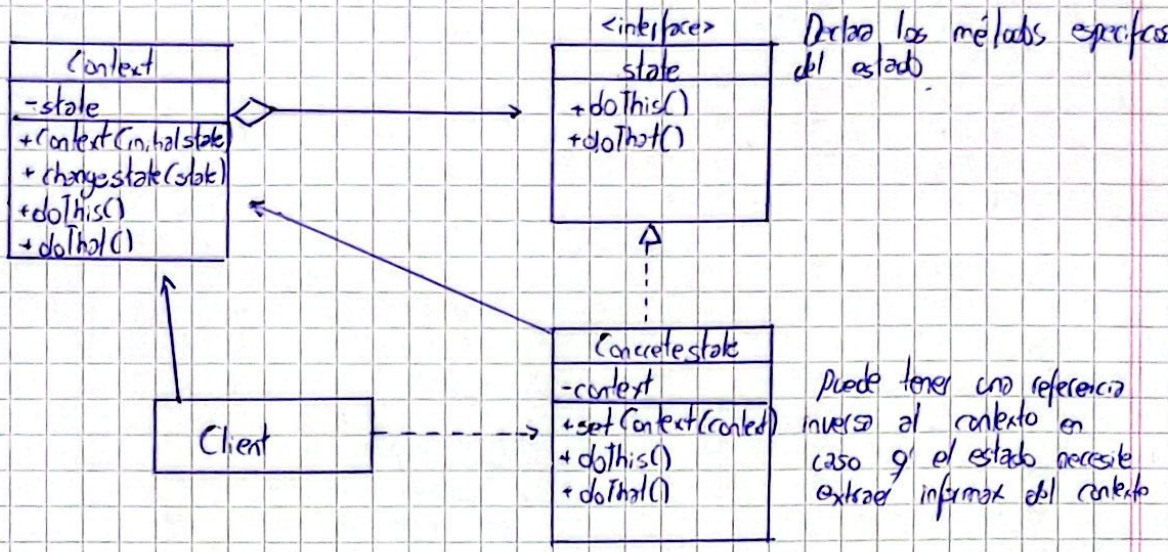
Strategy pattern: permite definir una familia de algoritmos, colocar cada uno en una clase separada y hacer sus objetos intercambiables. C/ el objetivo de desacoplar un algoritmo del objeto q' lo usa, cambia el algoritmo q' usa dinámica'.
Necesita uno de diferentes algoritmos comunales p/ realizar una misma tarea. Es útil p/ permitir q' una clase durante la ejecux delegue un algoritmo a una instancia de varias estrategias.

State pattern: permite a un objeto alterar su comportamiento cuando su estado interno cambia
El comportamiento depende del estado en q' se encuentre el objeto. Es útil p/ una clase q' debe realizar transiciones e/ estados fácil.

En state los <> estados son internos al contexto y las clases NO son elegidas x los clientes si no q' la transix se realiza e/ los mismos estados
En strategy las <> estrategias son conocidas desde fuera del contexto ya q' son setteadas x los clientes.



State


Context
-state
+Context(initialState)
+changeState(state)
+doThis()
+doThat()
<interface>
State
+doThis()
+doThat()
Declara los métodos específicos del estado
ConcreteState
-context
+setContext(context)
+doThis()
+doThat()
Puede tener una referencia inversa al contexto en caso q' el estado necesite extraer información del contexto
Client